MATRIXPOLICY: ROLE-AWARE OPTIMIZATION FOR RATIONAL TRANSFORMER NONLINEARITIES

Anonymous authors
Paper under double-blind review

ABSTRACT

Transformer nonlinear sublayers contain an input matrix, a coordinatewise nonlinearity, and an output matrix, but standard optimizers usually update these matrices without using their distinct functional roles. We introduce a global-rational Rational Latent Basis (RLB) sublayer that exposes normalized group coordinates, trainable real-pole-free P5/Q4 responses, derivative/output response summaries, and an idealized positive matrix-pair rescaling relation that becomes approximate under the implemented RMS floor. We then introduce MatrixPolicy, a role-aware optimizer for the RLB input and output matrices. In the reported configuration, rational coefficients and all non-RLB-matrix Transformer parameters use AdamW, while MatrixPolicy applies centered group-gradient redistribution, a transient gated Muon substep, and bounded matrix-pair rescaling only to the RLB matrices. The main empirical benefit is faster arrival at useful loss levels, measured primarily as the percentage of comparator target-arrival tokens saved. Across five 12-layer, width-768 language-model pretraining corpora, MatrixPolicy saves 19.5–22.8% versus SiLU with AdamW and 29.2–34.0% versus SiLU with Muon in E1; in E2 it saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with positive measured training-time savings for every dataset/comparator pair. TODO: Test whether these percentage target-arrival savings persist in a larger-scale experiment beyond the current fixed-scale setting.

1 INTRODUCTION

Transformer nonlinear sublayers are usually described by their scalar activation: GELU, SiLU, SwiGLU, or a related gated variant (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). That shorthand hides a larger optimization object. A sublayer first chooses coordinates through an input matrix, applies a nonlinear response in those coordinates, and then recombines the resulting features through an output matrix. The two matrices do not play the same role, yet AdamW, Muon, and other general optimizers usually see them as ordinary tensors with no knowledge of which side of the nonlinearity they occupy.

We study whether the nonlinear sublayer can expose enough structure for the optimizer to use. The key idea is to make the activation an interface, not only a pointwise function. If the sublayer provides normalized groups, a trainable rational response in each group, observable derivative and curve-response scales, and a controlled positive rescaling relation between the adjacent matrices, then the optimizer can assign different update behavior to the input and output roles without changing the rest of the Transformer.

We instantiate this idea with a global-rational Rational Latent Basis (RLB) sublayer. It projects the residual stream into grouped latent coordinates, RMS-normalizes each group, applies one trainable real-pole-free P5/Q4 response per group, and projects back to the model width. The main variant used in this paper has no active local atoms: there are no trainable atom centers and no atom coefficient parameters. This choice keeps the nonlinear response simple enough to audit while preserving the matrix-role signals that MatrixPolicy uses.

MatrixPolicy is the optimizer coupled to this sublayer. It is not a generic optimizer for every Transformer tensor. It acts only on the two RLB matrices, using batch-estimated rational gain proxies, relative gradient scales, and the matrix pair ratio to redistribute group updates, apply a transient

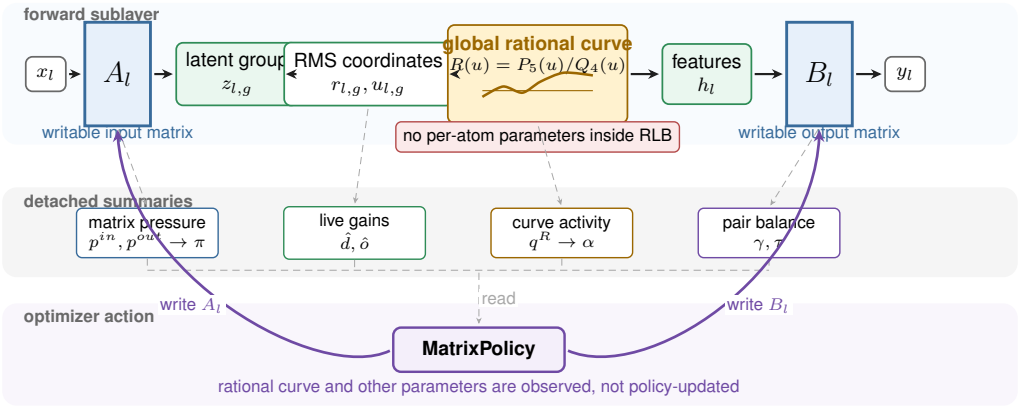

Figure 1: Global-rational RLB exposes an optimizer-visible interface. Solid arrows are the forward map $x_l \mapsto y_l$; dashed arrows are detached statistics read by MatrixPolicy; purple arrows are MatrixPolicy actions and terminate only at the RLB input/output matrices. The rational response is AdamW-trained and observed by the policy, but it is not a MatrixPolicy update target in the reported configuration.

matrix-direction substep, and keep the input/output representatives balanced. In the reported configuration, rational coefficients, attention weights, embeddings, and all other non-RLB-matrix parameters use AdamW.

We evaluate the method on a fixed 12-layer, width-768 causal Transformer across DCLM, FineWeb-Edu, FineWeb, Dolma-sample, and C4. E1 trains for 100M tokens, where early optimization differences remain visible before long-budget saturation compresses endpoints. In this regime MatrixPolicy reaches common validation-loss targets substantially earlier than AdamW and Muon comparators on both SiLU and global-rational RLB controls. E2 extends the same architecture to 300M tokens; endpoint gaps are smaller after longer training, but MatrixPolicy still keeps lower endpoint validation loss across all five datasets.

The contributions are:

- a global-rational RLB sublayer that exposes matrix roles, curve-gain summaries, and an idealized positive matrix-pair relation without local atom parameters;
- MatrixPolicy, a role-aware optimizer for the global-rational RLB input and output matrices in a reported configuration where rational coefficients and non-RLB-matrix parameters use AdamW;
- a matched empirical package showing faster E1 target arrival in both tokens and estimated training time against AdamW and Muon controls, with E2 saturation results showing that the endpoint advantage persists after longer training.

2 METHOD

RLB and MatrixPolicy are designed as an activation–optimizer pair. RLB defines the nonlinear sublayer and exposes the statistics; MatrixPolicy consumes those statistics to update only the adjacent input and output matrices. Throughout the method we use

$$\mathrm{clip}(x, a, b) = \min\{\max\{x, a\}, b\}, \tag{1}$$

$$\mathrm{smoothstep}(a, b, x) = 3\omega^2 - 2\omega^3, \qquad \omega = \mathrm{clip}\left(\frac{x-a}{b-a}, 0, 1\right). \tag{2}$$

2.1 RATIONAL LATENT BASIS SUBLAYER

For Transformer layer $l \in \{1, \dots, L\}$, let $x_l \in \mathbb{R}^d$ denote the input to the nonlinear sublayer. RLB first projects this input to a latent width $H = Gm$ and partitions the latent vector into G groups of width m:

$$z_l = A_l x_l, \qquad z_l = \operatorname{concat}_{g=1}^{G} z_{l,g}, \qquad z_{l,g} \in \mathbb{R}^m, \tag{3}$$

where $A_l \in \mathbb{R}^{H \times d}$ is the input-domain matrix. Each group is normalized by its root-mean-square radius,

$$r_{l,g} = \sqrt{m^{-1}\|z_{l,g}\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_{l,g} = z_{l,g}/r_{l,g}, \tag{4}$$

with numerical floor $\epsilon_{\mathrm{rms}} > 0$. A scalar response is applied coordinatewise in the normalized coordinates and then multiplied by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \qquad h_l = \operatorname{concat}_{g=1}^{G} h_{l,g}, \qquad y_l = B_l h_l, \tag{5}$$

where $B_l \in \mathbb{R}^{d \times H}$ is the output-recombination matrix.

The curve $R_{l,g}$ is a trainable real-pole-free piecewise-rational P5/Q4 response,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \qquad P_{l,g}(u) = \sum_{i=0}^{5} a_{l,g,i} u^i, \tag{6}$$

with denominator

$$Q_{l,g}(u) = 1 + |b_{l,g,1}|\,|u| + |b_{l,g,2}|\,u^2 + |b_{l,g,3}|\,|u|^3 + |b_{l,g,4}|\,u^4. \tag{7}$$

The absolute-value terms make the response piecewise rational rather than a single algebraic rational function of u, but they guarantee $Q_{l,g}(u) \geq 1$ on the real line. Derivative-gain statistics use the autodiff derivative of the implemented expression, with $\operatorname{sign}(0) = 0$; equations involving $R'_{l,g}$ are understood as almost-everywhere derivatives. Numerator and denominator coefficients are initialized by fitting SiLU on $[-5, 5]$ and are trained thereafter by AdamW in the reported configuration; MatrixPolicy only observes their gradients. The implementation uses the same interval for probe-grid gain summaries. In the main variant there are no active local Cauchy atoms, trainable atom centers, or atom coefficient parameters.

The normalization in equation 4 gives the optimizer two views of each group. The normalized coordinate $u_{l,g}$ determines where the response is evaluated, while the radius $r_{l,g}$ carries group scale back to the residual stream. For fixed curves $R_l = \{R_{l,g}\}_{g=1}^{G}$, define the RLB block map

$$f_l(x; A_l, B_l, R_l) = B_l \operatorname{concat}_{g=1}^{G} r_{l,g} R_{l,g}(u_{l,g}), \tag{8}$$

where $z_l = A_l x$, $z_{l,g}$, $r_{l,g}$, and $u_{l,g}$ are computed by equation 3–equation 4. In the idealized case $\epsilon_{\mathrm{rms}} = 0$ with nonzero group radii, this construction admits the positive rescaling

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a) A_l, B_l D_l(a)^{-1}, R_l), \tag{9}$$

where $D_l(a) = \operatorname{diag}(a_1 I_m, \dots, a_G I_m)$ for $a \in \mathbb{R}^G_{>0}$. MatrixPolicy uses this relation only for small, bounded balancing steps because the additive RMS floor makes the block-map identity approximate, and finite optimizer-state handling makes subsequent dynamics only approximately invariant in the implemented system.

2.2 ROLE-AWARE OPTIMIZER STATISTICS

MatrixPolicy tracks role-specific relative gradient scales and curve-gain summaries for each layer and group. For any tensor V, define

$$\operatorname{rms}(V) = \sqrt{\operatorname{mean}(V^2)}, \qquad \operatorname{rms}_\epsilon(V) = \sqrt{\operatorname{mean}(V^2) + \epsilon}. \tag{10}$$

The constants $\epsilon_p > 0$ and $\epsilon_g > 0$ denote fixed additive floors in squared-RMS units. The resulting pressure and activity scores are fixed-implementation control signals rather than reparameterization-invariant scalar observables. Let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l, and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l. The relative matrix-gradient scales are

$$p_{l,g}^{\mathrm{in}} = \frac{\operatorname{rms}_{\epsilon_p}(\nabla A_{l,g})}{\operatorname{rms}_{\epsilon_p}(A_{l,g})}, \qquad p_{l,g}^{\mathrm{out}} = \frac{\operatorname{rms}_{\epsilon_p}(\nabla B_{l,g})}{\operatorname{rms}_{\epsilon_p}(B_{l,g})}. \tag{11}$$

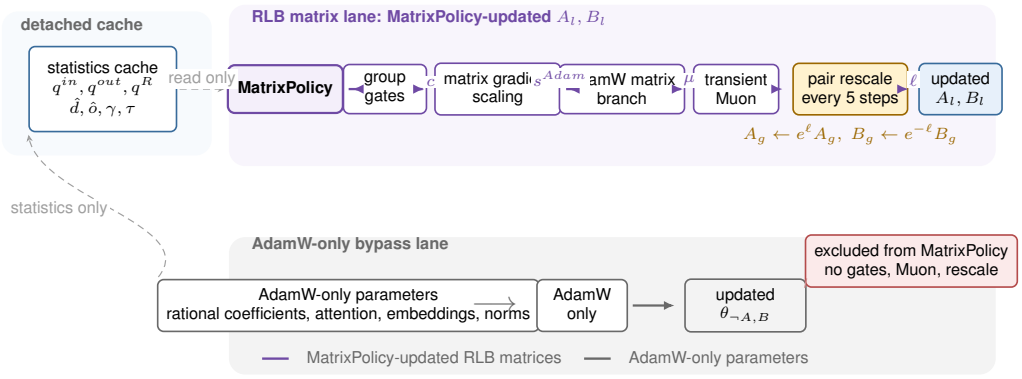

Figure 2: Reported MatrixPolicy step. Policy statistics produce bounded gates; non-module quantities such as $c_{l,g,\sigma}$, $\mu_{l,\sigma,t}$, $\gamma_{l,g}$, $\tau_{l,g}$, and $\ell_{l,g,t}$ are control labels on the corresponding connectors. Non-matrix parameters, including rational coefficients, stay on the AdamW-only lane and receive no MatrixPolicy gate, Muon, or pair-rescale action.

The coefficient-gradient energy scale for the global response is

$$p^R_{l,g} = \left[\frac{1}{2}\left(\mathrm{mean}((\nabla a_{l,g,\cdot})^2) + \mathrm{mean}((\nabla b_{l,g,\cdot})^2)\right) + \epsilon_p\right]^{1/2}. \tag{12}$$

MatrixPolicy maintains exponential moving averages $q^{\mathrm{in}}_{l,g}$, $q^{\mathrm{out}}_{l,g}$, and $q^R_{l,g}$ of these quantities with decay 0.95, and forms

$$\pi_{l,g} = \log q^{\mathrm{in}}_{l,g} - \log q^{\mathrm{out}}_{l,g}, \qquad \alpha_{l,g} = \log q^R_{l,g} - \frac{1}{2}\left(\log q^{\mathrm{in}}_{l,g} + \log q^{\mathrm{out}}_{l,g}\right). \tag{13}$$

Here $\pi_{l,g}$ compares the update scale of the two adjacent matrix roles. The score $\alpha_{l,g}$ is a heuristic coefficient-activity statistic: because $p^R_{l,g}$ is an average of numerator- and denominator-gradient energies rather than a dimensionless matrix pressure, MatrixPolicy uses $\alpha_{l,g}$ only inside bounded redistribution and gating rules.

The activation also provides batch-estimated curve gains. From a recent sample of normalized coordinates $u_{l,g}$ cached by the RLB forward path, MatrixPolicy uses floored summaries

$$\hat{d}^{\mathrm{live}}_{l,g} = \sqrt{\mathrm{mean}\left(R'_{l,g}(u_{l,g})^2\right) + \epsilon_g}, \qquad \hat{o}^{\mathrm{live}}_{l,g} = \sqrt{\mathrm{mean}(R_{l,g}(u_{l,g})^2) + \epsilon_g}. \tag{14}$$

In equation 14, the mean is over all cached scalar normalized coordinates for group (l, g) across batch, sequence positions, and the m group coordinates. The derivative gain is a proxy for the input-domain role. The output gain is a curve-response proxy for the recombination role; it does not include the group radius $r_{l,g}$ and therefore is not the full feature scale seen by B_l. Both gains are deliberately cheap summaries rather than full block-Jacobian preconditioners.

2.3 MatrixPolicy Update Rule

Let t be the optimizer step, T the planned number of training steps, and $\xi_t = t/T$. A smooth gate

$$\phi_t = \mathrm{smoothstep}(0.02, 0.30, \xi_t) \tag{15}$$

turns on the group-gradient policy. For positive group values v_j, define $\mathrm{geomean}^G_{j=1}(v_j) = \exp\left(G^{-1}\sum_{j=1}^G \log v_j\right)$. The multiplier for group g and role $\sigma \in \{\mathrm{in}, \mathrm{out}\}$ is

$$c_{l,g,\sigma} = c^{\mathrm{gain}}_{l,g,\sigma} c^{\mathrm{pressure}}_{l,g,\sigma} c^{\mathrm{activity}}_{l,g}. \tag{16}$$

For the input role, the gain term uses $\hat{d}^{\mathrm{live}}_{l,g}$; for the output role, it uses $\hat{o}^{\mathrm{live}}_{l,g}$:

$$c^{\mathrm{gain}}_{l,g,\mathrm{in}} = \left(\frac{\mathrm{geomean}^G_{j=1}\,\hat{d}^{\mathrm{live}}_{l,j}}{\hat{d}^{\mathrm{live}}_{l,g}}\right)^{0.20\phi_t}, \qquad c^{\mathrm{gain}}_{l,g,\mathrm{out}} = \left(\frac{\mathrm{geomean}^G_{j=1}\,\hat{o}^{\mathrm{live}}_{l,j}}{\hat{o}^{\mathrm{live}}_{l,g}}\right)^{0.20\phi_t}. \tag{17}$$

With $\tilde{\pi}_{l,g} = \mathrm{clip}(\pi_{l,g}, -1.50, 1.50)$, the relative-gradient term sets role-dependent group multipliers before the later role-wise recentering:

$$c_{l,g,\mathrm{in}}^{\mathrm{pressure}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \qquad c_{l,g,\mathrm{out}}^{\mathrm{pressure}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \tag{18}$$

Coefficient-gradient activity downweights groups whose rational coefficients are moving more strongly than the adjacent matrices before role-wise recentering:

$$c_{l,g}^{\mathrm{activity}} = \exp\left[-0.20\phi_t \,\mathrm{ReLU}\left(\frac{\alpha_{l,g} - 0.05}{0.45}\right)\right]. \tag{19}$$

The final multiplier is recentered and clipped,

$$c_{l,g,\sigma} \leftarrow \frac{c_{l,g,\sigma}}{\mathrm{geomean}_{j=1}^{G}\, c_{l,j,\sigma}}, \qquad c_{l,g,\sigma} \leftarrow \mathrm{clip}(c_{l,g,\sigma}, 0.75, 1.35), \tag{20}$$

so the policy redistributes update scale across groups, separately for each matrix role, without changing the role-wise mean log multiplier before clipping.

The scaled matrix gradients are

$$\widetilde{\nabla} A_{l,g} = c_{l,g,\mathrm{in}} \nabla A_{l,g}, \qquad \widetilde{\nabla} B_{l,g} = c_{l,g,\mathrm{out}} \nabla B_{l,g}. \tag{21}$$

After this scaling, the RLB matrices are updated by a role/depth AdamW branch and a transient Muon branch. Define $\delta_l = (l-1)/(L-1)$ for $L > 1$ and $\delta_l = 1/2$ otherwise. The role factors are

$$\varrho_{\mathrm{in}}(l) = \mathrm{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \qquad \varrho_{\mathrm{out}}(l) = \mathrm{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40), \tag{22}$$

and the AdamW matrix multiplier for role σ is

$$s_{l,\sigma}^{\mathrm{Adam}} = \mathrm{clip}\left(3.0 \max\{0.10, 1 + 1.20(\varrho_\sigma(l) - 1)\}, 0.40, 4.0\right). \tag{23}$$

The Muon branch is concentrated early in training and is reduced when relative scale imbalance or coefficient-gradient scale is high. Define

$$\Psi_{l,t} = \mathrm{clip}\left(\frac{1}{G}\sum_{g=1}^{G} |\pi_{l,g}|, 0, 1.50\right), \qquad \Omega_{l,t} = \mathrm{ReLU}\left(\frac{G^{-1}\sum_{g=1}^{G} \alpha_{l,g} - 0.05}{0.45}\right), \tag{24}$$

$$\psi_{l,t} = \mathrm{clip}\left(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15\right). \tag{25}$$

The branch gate is

$$\mu_{l,\sigma,t} = \mathrm{clip}\left(0.75\ \mathrm{smoothstep}(0.02, 0.12, \xi_t)[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)]\varrho_\sigma(l)\psi_{l,t}, 0, 0.75\right). \tag{26}$$

For $M_{l,\mathrm{in}} = A_l$ and $M_{l,\mathrm{out}} = B_l$, MatrixPolicy applies an AdamW matrix step followed by any active Muon substep. The gate $\mu_{l,\sigma,t}$ scales the two learning rates; it is not a convex mixing coefficient for a single update direction:

$$\begin{aligned} \widetilde{M}_{l,\sigma}^{t+1} &= \mathrm{AdamW}_{\eta_t s_{l,\sigma}^{\mathrm{Adam}}(1-\mu_{l,\sigma,t})}(M_{l,\sigma}^t; \widetilde{\nabla} M_{l,\sigma}^t), \\ M_{l,\sigma}^{t+1} &= \mathrm{Muon}_{\eta_t \mu_{l,\sigma,t}}(\widetilde{M}_{l,\sigma}^{t+1}; \widetilde{\nabla} M_{l,\sigma}^t), \end{aligned} \tag{27}$$

Equation equation 27 is operational update notation: AdamW and Muon carry separate optimizer states, their configured weight-decay and momentum terms are handled by the respective optimizer implementations, and both substeps use the gradient from training step t. Here η_t is the base learning rate and Muon denotes the `torch.optim.Muon` update on two-dimensional RLB matrix parameters: momentum 0.95, five Newton–Schulz orthogonalization steps, and the match-RMS learning-rate adjustment of the Muon optimizer (Liu et al., 2025). When all Muon fractions have decayed to zero, this branch is inactive.

The last operation is a bounded positive rescaling of the two matrix blocks around each rational curve. It is evaluated every five optimizer steps; on other steps it is skipped. Let $\mathcal{U} = \{u_k\}_{k=1}^{129}$ be a uniform probe grid over $[-5, 5]$, and define $\mathrm{rms}_{\mathcal{U}}(\varphi) = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} \varphi(u_k)^2}$. The fixed-grid

probe gains are diagnostics of the curve shape, not estimates of the current feature distribution; live gains in equation 14 provide the batch-distribution summaries. The probe gains are

$$\hat{d}_{l,g}^{\text{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R'_{l,g}(u_k)^2 + \epsilon_g}, \qquad \hat{o}_{l,g}^{\text{probe}} = \sqrt{|\mathcal{U}|^{-1} \sum_{u_k \in \mathcal{U}} R_{l,g}(u_k)^2 + \epsilon_g}, \tag{28}$$

and the current matrix log ratio is computed with floored matrix RMS values,

$$\gamma_{l,g} = \log \text{rms}_{\epsilon_p}(A_{l,g}) - \log \text{rms}_{\epsilon_p}(B_{l,g}). \tag{29}$$

The heuristic target ratio combines curve-gain and relative-gradient terms,

$$\tau_{l,g} = \frac{1}{2}\left(\log \hat{d}_{l,g}^{\text{probe}} - \log \hat{o}_{l,g}^{\text{probe}}\right) + 0.25\,\bar{\pi}_{l,g}, \qquad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \tag{30}$$

Coefficient-gradient scale modulates the rescaling strength through

$$a_{l,g}^{\text{rs}} = \exp(0.10\,\bar{\alpha}_{l,g}), \qquad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \tag{31}$$

The positive sign is intentional: high coefficient activity damps the group-gradient and Muon paths, but lets the bounded balancing step move slightly more quickly toward the representative ratio; the later log-step clip limits this effect. With schedule

$$s_{l,t} = 0.50\ \text{smoothstep}(0.03, 0.35, \xi_t)\ \text{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \tag{32}$$

the active-step log move is

$$\ell_{l,g,t} = \text{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}] s_{l,t} a_{l,g}^{\text{rs}}, -0.030, 0.030\right). \tag{33}$$

The matrix blocks are then updated as

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \tag{34}$$

The implementation applies the corresponding group scale to optimizer-state buffers as well, using squared scales for square-average state entries.

3 Related Work

Transformer nonlinear sublayers are often identified by their scalar response, from GELU and Swish/SiLU to gated variants such as GLU and SwiGLU (Hendrycks & Gimpel, 2016; Ramachandran et al., 2017; Shazeer, 2020). RLB follows the same architectural location but changes the interface exposed to optimization: instead of replacing only a fixed scalar curve, it exposes grouped coordinates, trainable rational responses, and the adjacent matrix roles used by MatrixPolicy.

Rational activations are motivated by the ability of low-degree rational functions to represent sharp transitions, saturation, and curved response shapes with few parameters. Recent approximation-theoretic work argues that trainable low-degree rational activations can have expressivity advantages over common fixed activations (Tang & Townsend, 2026). We use that result as motivation for the activation family, not as a proof of MatrixPolicy. The empirical comparison separates the global-rational RLB sublayer from the role-aware optimizer by including RLB controls with AdamW, Muon, Lion, SOAP, CAME, and ScheduleFree-style optimizers.

Adam and AdamW remain standard Transformer pretraining optimizers (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). Other work changes the first-order rule while keeping a generic tensor interface, including Lion and cautious optimizers (Chen et al., 2023; Liang et al., 2026), or uses matrix structure and low-rank information, including Muon, Sophia, GaLore, SOAP, and Adam-mini (Liu et al., 2025; 2024; Zhao et al., 2024; Vyas et al., 2025; Zhang et al., 2025). CAME and schedule-free training modify optimizer state or schedules (Luo et al., 2023; Defazio et al., 2024). MatrixPolicy is closest in spirit to matrix-aware optimization, but in the reported configuration it applies role-specific rules only to the input and output matrices of RLB; rational coefficients and non-RLB-matrix Transformer parameters use AdamW. Broader optimizer coverage is available in recent surveys (Wen et al., 2026).

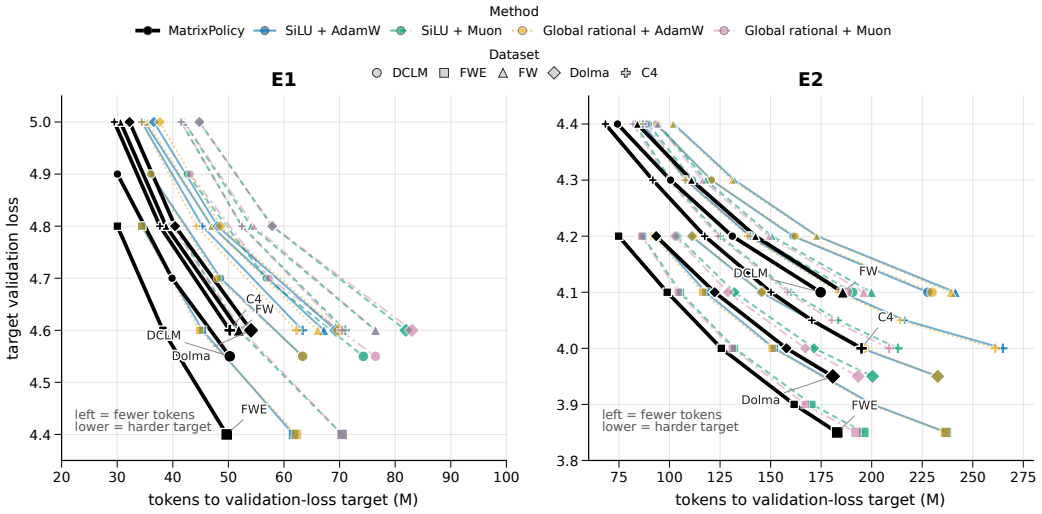

Figure 3: Target-arrival map for the completed fixed-scale study. Each point is the mean token count needed by one method to reach one validation-loss target on one dataset; connected points trace the same method and dataset from easier to harder targets. The horizontal coordinate is target-arrival tokens, and the vertical coordinate is the target validation loss, so curves further left reach the same loss with fewer tokens. The exact hard-target token, percentage, and time gaps are quantified in Table 1.

4 Experiments

The fixed-scale study uses the same 12-layer, width-768 causal Transformer on five corpora: DCLM/DataComp-LM (Li et al., 2024), FineWeb-Edu and FineWeb (Penedo et al., 2024), Dolma-sample (Soldaini et al., 2024), and C4 (Raffel et al., 2020). E1 trains for approximately 100M tokens and E2 for approximately 300M tokens, with three seeds per method/dataset pair. The main comparison keeps the architecture and data fixed while changing the activation and optimizer used in the nonlinear sublayer: MatrixPolicy with the global rational activation is compared with SiLU and global-rational activation controls under AdamW and Muon. Shared optimizer, evaluation-cadence, and throughput-measurement details are reported in Appendix A; broader optimizer controls are also reported there.

Table 1 is the primary empirical result. It asks: for the hardest common validation-loss target reached by every listed method in all three seeds, how many training tokens and how much measured training time are required? Across all five E1 datasets, MatrixPolicy saves 19.5–22.8% of the SiLU with AdamW target-arrival tokens and 29.2–34.0% of the SiLU with Muon target-arrival tokens. In E2, after longer training compresses endpoint gaps, MatrixPolicy still saves 22.3–26.4% versus SiLU with AdamW and 6.7–9.8% versus SiLU with Muon, with positive measured wall-clock savings in every listed comparison.

Figure 3 complements the table rather than repeating it. The table reports exact tokens, percentages, and measured minutes at the hardest common targets; the map shows the full ladder of common targets. Across both regimes and all five datasets, MatrixPolicy traces are consistently left of the SiLU and global-rational controls, so the hard-target table is not a single-threshold artifact but a compact accounting of the same target-arrival pattern.

The E1 curves show why target arrival is the right main readout. MatrixPolicy separates early in validation loss and validation perplexity while keeping training loss competitive. A long fixed budget eventually compresses many activation/optimizer endpoints, especially in E2, so the paper emphasizes the number of tokens and minutes needed to reach the same useful validation-loss levels rather than only the final loss after every method has consumed the full budget.

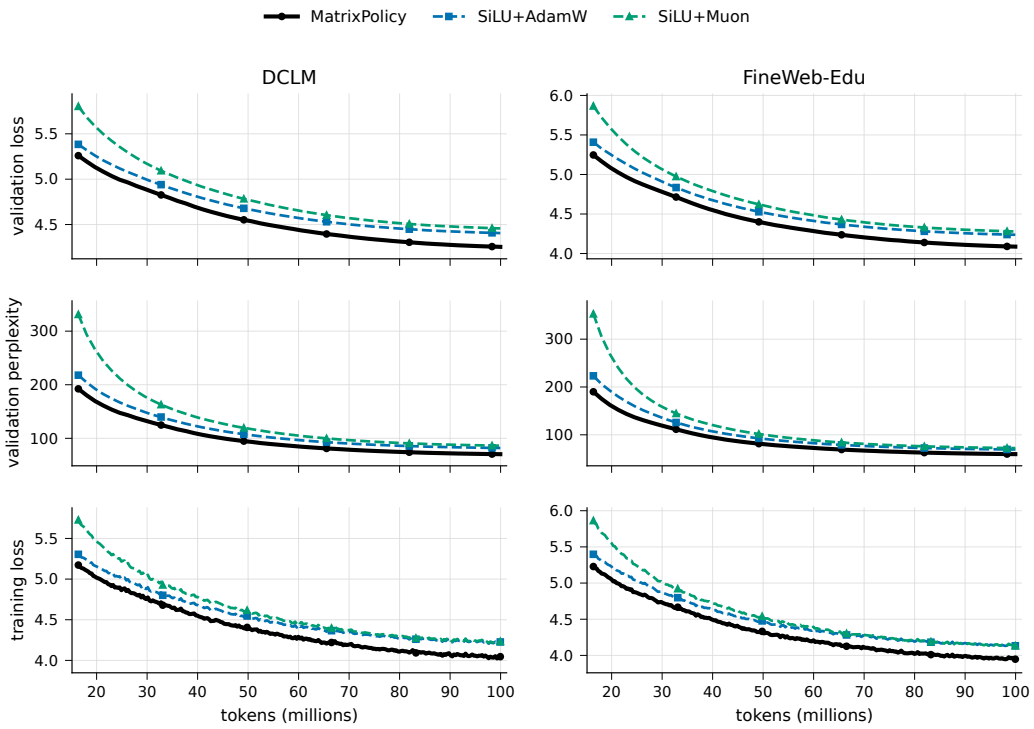

Figure 4: Representative E1 training dynamics on DCLM and FineWeb-Edu. The rows show validation loss, validation perplexity, and training loss. Lines are seed means over three runs and bands show one sample standard deviation. This figure uses the two standard SiLU baselines so that the early target-arrival effect in Table 1 remains visually legible.

Table 1: Main-paper target-arrival efficiency across the completed E1 and E2 datasets. For each dataset and target validation loss, the table reports the actual target-arrival tokens and wall-clock time for MatrixPolicy and each comparator. MatrixPolicy-at-target and comparator-at-target entries are ordered as tokens then time; all token entries are millions of tokens and all time entries are minutes. Tokens saved is comparator target tokens minus MatrixPolicy target tokens, with the percentage of comparator tokens saved in the final column.

Regime	Dataset	Target validation loss	Comparator	MatrixPolicy at target tokens and time	Comparator at target tokens and time	Tokens saved	Proportion saved
E1	DCLM	4.55	SiLU activation with AdamW	(50.2, 12.1)	(63.4, 20.2)	13.1	20.7%
E1	DCLM	4.55	SiLU activation with Muon	(50.2, 12.1)	(74.3, 25.3)	24.0	32.4%
E1	DCLM	4.55	Global rational activation with AdamW	(50.2, 12.1)	(63.4, 13.4)	13.1	20.7%
E1	DCLM	4.55	Global rational activation with Muon	(50.2, 12.1)	(76.5, 17.9)	26.2	34.3%
E1	FineWeb-Edu	4.40	SiLU activation with AdamW	(49.7, 12.1)	(61.7, 19.7)	12.0	19.5%
E1	FineWeb-Edu	4.40	SiLU activation with Muon	(49.7, 12.1)	(70.5, 23.7)	20.8	29.5%
E1	FineWeb-Edu	4.40	Global rational activation with AdamW	(49.7, 12.1)	(62.3, 13.3)	12.6	20.2%
E1	FineWeb-Edu	4.40	Global rational activation with Muon	(49.7, 12.1)	(70.5, 17.0)	20.8	29.5%
E1	FineWeb	4.60	SiLU activation with AdamW	(51.9, 11.8)	(67.2, 15.1)	15.3	22.8%
E1	FineWeb	4.60	SiLU activation with Muon	(51.9, 11.8)	(76.5, 18.7)	24.6	32.1%
E1	FineWeb	4.60	Global rational activation with AdamW	(51.9, 11.8)	(66.1, 15.3)	14.2	21.5%
E1	FineWeb	4.60	Global rational activation with Muon	(51.9, 11.8)	(76.5, 19.0)	24.6	32.1%
E1	Dolma	4.60	SiLU activation with AdamW	(54.1, 12.1)	(69.4, 16.6)	15.3	22.0%
E1	Dolma	4.60	SiLU activation with Muon	(54.1, 12.1)	(81.9, 20.9)	27.9	34.0%
E1	Dolma	4.60	Global rational activation with AdamW	(54.1, 12.1)	(69.9, 16.3)	15.8	22.7%
E1	Dolma	4.60	Global rational activation with Muon	(54.1, 12.1)	(83.0, 21.0)	28.9	34.9%
E1	C4	4.60	SiLU activation with AdamW	(50.2, 11.0)	(63.4, 13.5)	13.1	20.7%
E1	C4	4.60	SiLU activation with Muon	(50.2, 11.0)	(71.0, 16.5)	20.8	29.2%
E1	C4	4.60	Global rational activation with AdamW	(50.2, 11.0)	(62.3, 13.9)	12.0	19.3%
E1	C4	4.60	Global rational activation with Muon	(50.2, 11.0)	(70.5, 17.8)	20.2	28.7%
E2	DCLM	4.10	SiLU activation with AdamW	(174.8, 37.2)	(227.7, 51.1)	53.0	23.3%
E2	DCLM	4.10	SiLU activation with Muon	(174.8, 37.2)	(190.6, 42.4)	15.8	8.3%
E2	DCLM	4.10	Global rational activation with AdamW	(174.8, 37.2)	(229.9, 51.5)	55.2	24.0%
E2	DCLM	4.10	Global rational activation with Muon	(174.8, 37.2)	(187.9, 42.6)	13.1	7.0%
E2	FineWeb-Edu	3.85	SiLU activation with AdamW	(183.0, 37.8)	(237.0, 55.9)	54.1	22.8%
E2	FineWeb-Edu	3.85	SiLU activation with Muon	(183.0, 37.8)	(196.1, 44.8)	13.1	6.7%
E2	FineWeb-Edu	3.85	Global rational activation with AdamW	(183.0, 37.8)	(236.5, 49.3)	53.5	22.6%
E2	FineWeb-Edu	3.85	Global rational activation with Muon	(183.0, 37.8)	(192.2, 47.4)	9.3	4.8%
E2	FineWeb	4.10	SiLU activation with AdamW	(185.7, 38.5)	(241.4, 57.0)	55.7	23.1%
E2	FineWeb	4.10	SiLU activation with Muon	(185.7, 38.5)	(199.9, 44.4)	14.2	7.1%
E2	FineWeb	4.10	Global rational activation with AdamW	(185.7, 38.5)	(239.2, 52.3)	53.5	22.4%
E2	FineWeb	4.10	Global rational activation with Muon	(185.7, 38.5)	(196.1, 44.0)	10.4	5.3%
E2	Dolma	3.95	SiLU activation with AdamW	(180.8, 41.1)	(232.7, 46.9)	51.9	22.3%
E2	Dolma	3.95	SiLU activation with Muon	(180.8, 41.1)	(200.4, 48.1)	19.7	9.8%
E2	Dolma	3.95	Global rational activation with AdamW	(180.8, 41.1)	(232.7, 47.4)	51.9	22.3%
E2	Dolma	3.95	Global rational activation with Muon	(180.8, 41.1)	(193.3, 44.7)	12.6	6.5%
E2	C4	4.00	SiLU activation with AdamW	(195.0, 39.4)	(264.9, 60.7)	69.9	26.4%
E2	C4	4.00	SiLU activation with Muon	(195.0, 39.4)	(213.0, 51.7)	18.0	8.5%
E2	C4	4.00	Global rational activation with AdamW	(195.0, 39.4)	(261.1, 56.0)	66.1	25.3%
E2	C4	4.00	Global rational activation with Muon	(195.0, 39.4)	(208.6, 48.8)	13.7	6.5%

Targets are the hardest pre-specified validation-loss levels reached by MatrixPolicy and every listed comparator in all three seeds. Target arrivals use the native 50-step evaluation cadence, equal to 1.64 million tokens per readout. Wall-clock times use cleaned per-dataset/method throughput summaries from completed runs.

5 TODO: Larger-Scale Training Results

TODO: This section is reserved for larger-scale training results. The current claim is limited to the completed fixed-scale E1/E2 evidence in Table 1 and Figures 3–4.

6 TODO: Component Ablations

TODO: This section is reserved for component ablations isolating the centered group redistribution, transient Muon gate, bounded pair rescaling, and the global rational activation interface.

7 Conclusion

Global-rational RLB turns the nonlinear Transformer sublayer into an optimizer-visible interface: it exposes grouped normalized coordinates, trainable global rational responses, and an idealized positive matrix-pair relation used as a bounded balancing heuristic. MatrixPolicy uses that interface to update the RLB input and output matrices with role-aware statistics, while rational coefficients and non-RLB-matrix parameters use AdamW. The completed E1/E2 runs show a consistent target-arrival pattern: MatrixPolicy reaches common validation losses with fewer tokens and less measured training time than the listed SiLU and global-rational controls. Larger-scale training and component ablations remain explicit TODO sections in the current draft.

References

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL https://neurips.cc/virtual/2024/poster/96925.

Dan Hendrycks and Kevin Gimpel. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL https://mlanthology.org/iclr/2015/kingma2015iclr-adam/.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.

Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=zBPZeRjfgu.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is Scalable for LLM Training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL https://aclanthology.org/2023.acl-long.243/.

Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for Activation Functions. *arXiv preprint arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.

Noam Shazeer. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *arXiv preprint arXiv:2402.00159*, 2024. URL https://arxiv.org/abs/2402.00159.

Maosen Tang and Alex Townsend. Rational Neural Networks have Expressivity Advantages. *arXiv preprint arXiv:2602.12390*, 2026. doi: 10.48550/arXiv.2602.12390. URL https://arxiv.org/abs/2602.12390.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.

Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=2J51qUZ0iG.

Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=hYHsrKDiX7.

Table 2: E1 target-arrival savings for activation and optimizer controls. Each row compares MatrixPolicy with one activation/optimizer control at the hardest common validation-loss target reached by all listed methods on that dataset.

Dataset	Target validation loss	MatrixPolicy target tokens mean and standard deviation	Comparator	Tokens saved	Proportion saved	Time saved (minutes)
DCLM	4.55	50.2±0.9	SiLU activation with AdamW	13.1	20.7%	8.0
DCLM	4.55	50.2±0.9	Global rational activation with AdamW	13.1	20.7%	1.2
DCLM	4.55	50.2±0.9	Global rational activation with Muon	26.2	34.3%	5.8
FineWeb-Edu	4.40	49.7±0.9	SiLU activation with AdamW	12.0	19.5%	7.6
FineWeb-Edu	4.40	49.7±0.9	Global rational activation with AdamW	12.6	20.2%	1.2
FineWeb-Edu	4.40	49.7±0.9	Global rational activation with Muon	20.8	29.5%	4.9
FineWeb	4.60	51.9±0.9	SiLU activation with AdamW	15.3	22.8%	3.4
FineWeb	4.60	51.9±0.9	Global rational activation with AdamW	14.2	21.5%	3.6
FineWeb	4.60	51.9±0.9	Global rational activation with Muon	24.6	32.1%	7.2
Dolma	4.60	54.1±0.0	SiLU activation with AdamW	15.3	22.0%	4.5
Dolma	4.60	54.1±0.0	Global rational activation with AdamW	15.8	22.7%	4.2
Dolma	4.60	54.1±0.0	Global rational activation with Muon	28.9	34.9%	8.9
C4	4.50	59.0±1.6	SiLU activation with AdamW	21.3	26.5%	4.3
C4	4.50	59.0±1.6	Global rational activation with AdamW	20.2	25.5%	4.9
C4	4.50	59.0±1.6	Global rational activation with Muon	31.7	34.9%	10.0

Target arrivals are read at the native 50-step evaluation cadence from the completed E1 JSONL logs. Time estimates use cleaned per-dataset/method training-throughput summaries. Token entries are millions of tokens.

A Experimental Details and Supporting Results

All runs use a 12-layer causal Transformer with width 768, 12 attention heads, sequence length 256, RLB latent width 3072, and 32768 global tokens per optimizer step. E1 consists of 3050 optimizer steps, approximately 100M tokens, and E2 consists of 9150 optimizer steps, approximately 300M tokens. Each method/dataset pair is run with seeds 1337, 2027, and 3407. Token-to-target readouts use evaluation events every 50 steps, so arrivals are quantized in increments of 1.64M tokens.

The MatrixPolicy rows in the main table use the global-rational RLB activation with no local atoms: the active nonlinearity is only the group-global P5/Q4 response in equation 6–equation 7. The reported configuration uses AdamW as the backbone optimizer and disables the separate rational-coefficient optimizer. It enables MatrixPolicy group redistribution with gain/pressure/activity strengths $0.20/0.10/0.20$, schedule window 0.02–0.30, and group multiplier clip $[0.75, 1.35]$. The bounded RLB pair-rescale wrapper runs every five optimizer steps with covariant optimizer state scaling. Rational coefficients, attention weights, embeddings, normalization parameters, and ordinary Transformer weights use AdamW in this configuration.

Table 2 adds RLB optimizer controls for the E1 regime. It uses the hardest pre-specified target reached by MatrixPolicy and all listed comparators in all three seeds. These rows separate the activation from the MatrixPolicy update: training the same global-rational activation with AdamW or Muon does not recover the target-arrival frontier reached by MatrixPolicy.

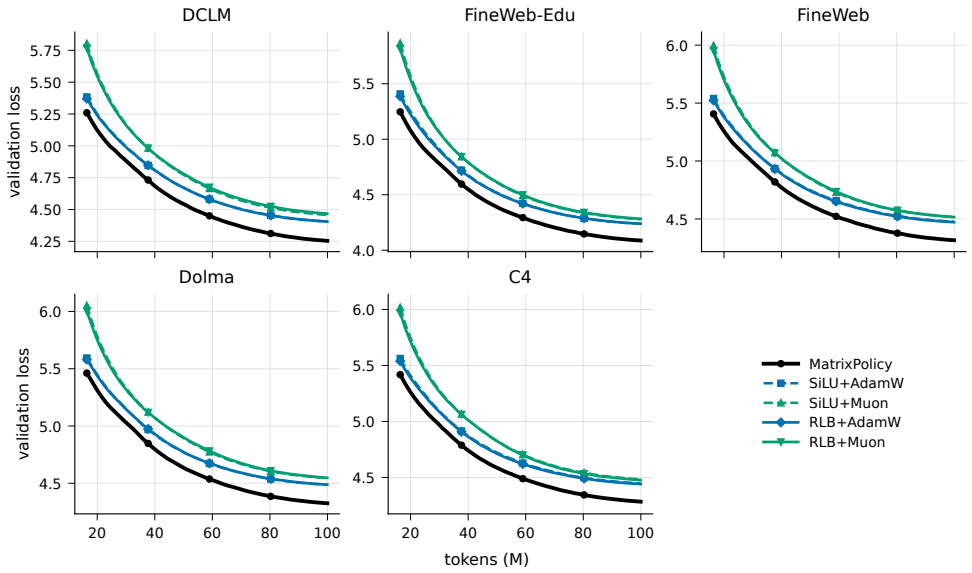

Figure 5: E1 validation-loss trajectories for MatrixPolicy, SiLU baselines, and RLB optimizer controls. Lines are seed means and bands show one sample standard deviation.

Table 3: Endpoint validation-loss gaps relative to MatrixPolicy across the five datasets. Positive gaps mean MatrixPolicy has lower final validation loss.

Comparator	E1 mean gap	E1 minimum gap	E2 mean gap	E2 minimum gap
Global rational activation with Lion	0.043	0.037	0.036	0.034
SiLU activation with Lion	0.065	0.062	0.042	0.039
Global rational activation with Muon	0.205	0.193	0.039	0.036
SiLU activation with Muon	0.203	0.191	0.047	0.044
Global rational activation with AdamW	0.156	0.151	0.099	0.098
SiLU activation with AdamW	0.157	0.150	0.100	0.098
Global rational activation with SOAP	0.220	0.162	0.125	0.109
SiLU activation with SOAP	0.171	0.162	0.153	0.145
Global rational activation with ScheduleFree optimizer	0.687	0.633	0.422	0.406
SiLU activation with ScheduleFree optimizer	0.706	0.649	0.430	0.409
Global rational activation with CAME	0.812	0.760	0.498	0.460
SiLU activation with CAME	0.817	0.757	0.441	0.416

B RLB Calculus and Matrix-Role Structure

B.1 Real-pole-free group-global response

For one layer and group, write

$$P(u) = \sum_{i=0}^{5} a_i u^i, \qquad Q(u) = 1 + \sum_{j=1}^{4} |b_j| \phi_j(u), \qquad \phi(u) = (|u|, u^2, |u|^3, u^4). \tag{35}$$

Since $\phi_j(u) \geq 0$ for all real u, $Q(u) \geq 1$ and $R(u) = P(u)/Q(u)$ has no real pole. Away from $u = 0$ for input derivatives,

$$R'(u) = \frac{P'(u)Q(u) - P(u)Q'(u)}{Q(u)^2}, \tag{36}$$

where

$$P'(u) = \sum_{i=1}^{5} i a_i u^{i-1}, \qquad Q'(u) = |b_1| \operatorname{sign}(u) + 2|b_2| u + 3|b_3| u|u| + 4|b_4| u^3. \tag{37}$$

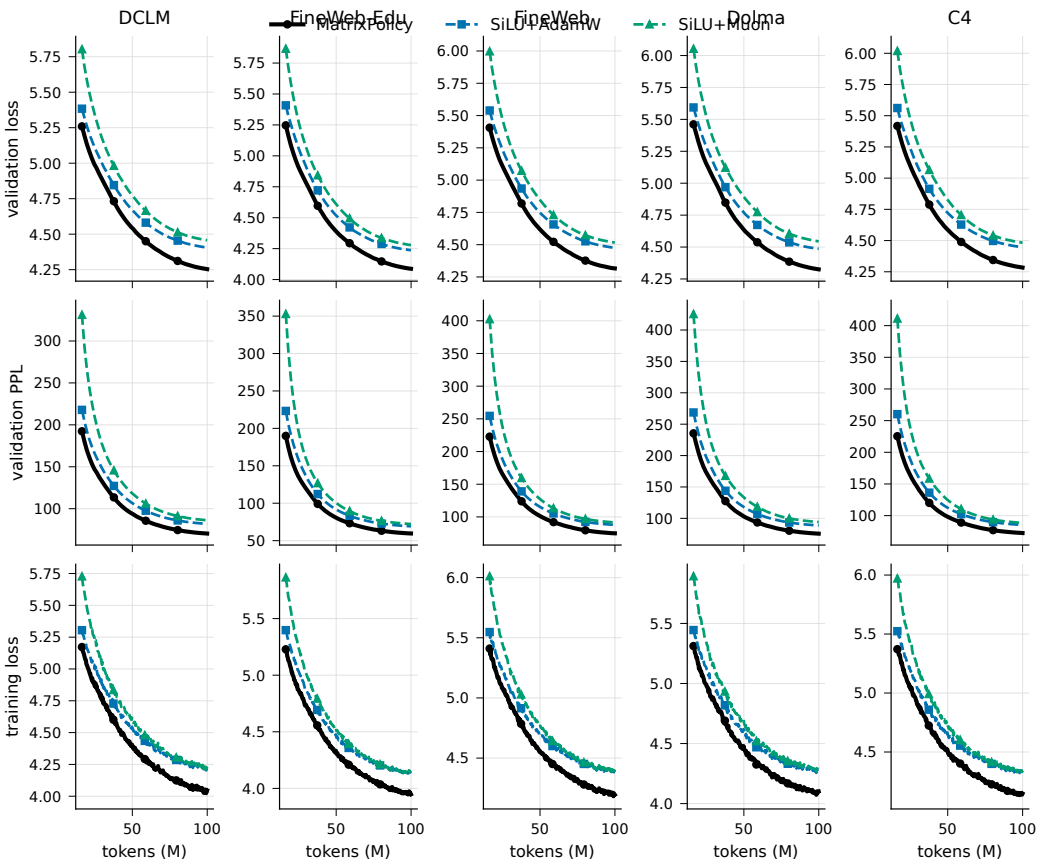

Figure 6: Full E1 multi-metric trajectories across all five datasets for the main SiLU comparison. Rows show validation loss, validation perplexity, and training loss; columns show datasets. Lines are seed means and bands show one sample standard deviation.

The implementation uses the autodiff convention $\operatorname{sign}(0) = 0$ for derivative-gain statistics. The coefficient features are explicit:

$$\frac{\partial R(u)}{\partial a_i} = \frac{u^i}{Q(u)}, \qquad \frac{\partial R(u)}{\partial b_j} = -\frac{P(u)}{Q(u)^2} \operatorname{sign}(b_j)\phi_j(u), \tag{38}$$

with the usual autodiff subgradient convention at $b_j = 0$. These derivatives make output-factor and derivative gains directly observable from the layer/group global curve, without introducing local-atom parameters.

B.2 Normalized-group Jacobian and gain summaries

For a group vector $z \in \mathbb{R}^m$, define

$$r(z) = \sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}, \qquad u(z) = z/r(z), \qquad h(z) = r(z)R(u(z)), \tag{39}$$

where R is applied coordinatewise. The radius derivative is

$$\mathrm{d}r = \frac{z^\top \mathrm{d}z}{mr} = \frac{u^\top \mathrm{d}z}{m}, \tag{40}$$

and the normalized-coordinate derivative is

$$\mathrm{d}u = \frac{1}{r}\left(I - \frac{uu^\top}{m}\right)\mathrm{d}z. \tag{41}$$

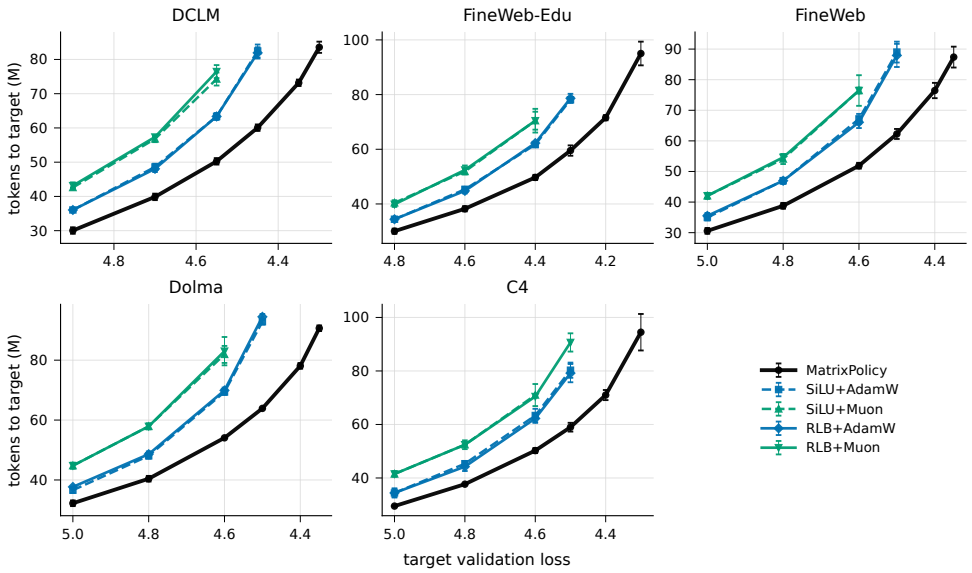

Figure 7: E1 target-arrival frontiers over the pre-specified validation-loss target grid. Lower token counts at the same target indicate faster arrival; vertical bars show one sample standard deviation across three seeds.

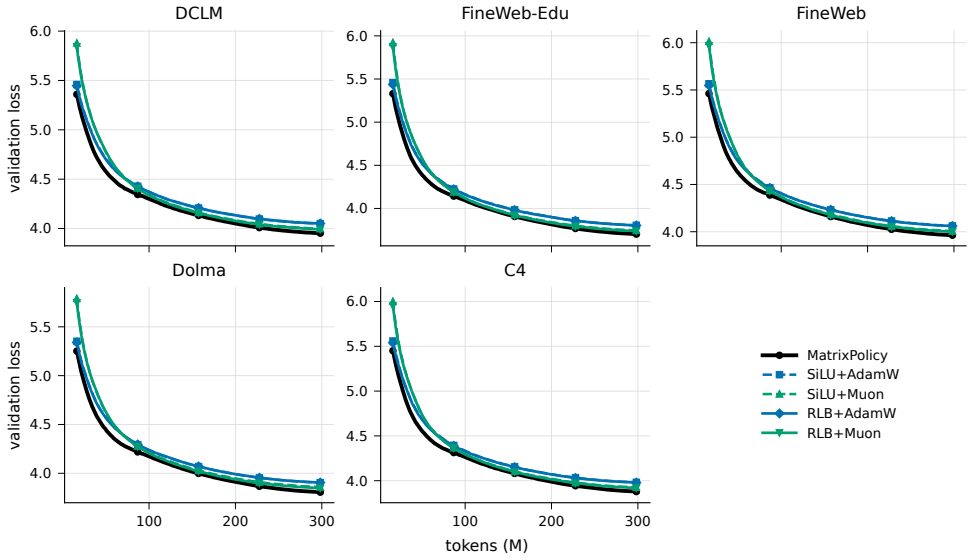

Figure 8: E2 validation-loss trajectories for the same comparator set used in the appendix E1 control figures. These longer-budget curves support the saturation discussion in Section 4.

Therefore the group Jacobian is

$$J_h(z) := \frac{\partial h}{\partial z} = R(u)\frac{u^\top}{m} + \mathrm{diag}(R'(u))\left(I - \frac{uu^\top}{m}\right). \tag{42}$$

This decomposition motivates separate output-gain and derivative-gain summaries. The vector $R(u)$ controls the curve-response factor inside the realized feature $h = rR(u)$, while $R'(u)$ appears in the tangent component of the input Jacobian seen by A_l. The summaries $\hat{o}$ and $\hat{d}$ are not full feature-scale or input-Jacobian measurements: the full feature also depends on r, and the full input gradient also depends on the radial $R(u)u^\top/m$ term and on $B_{l,g}$.

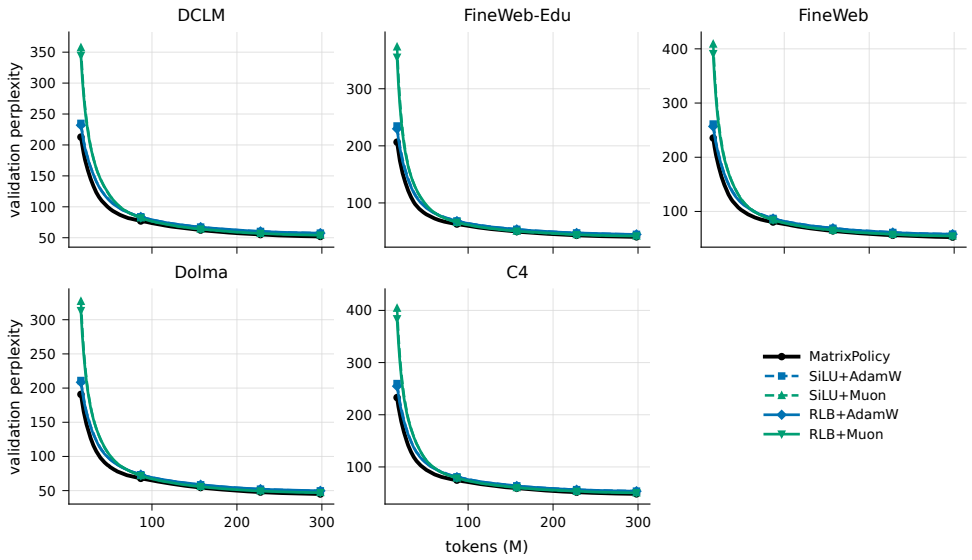

Figure 9: E2 validation-perplexity trajectories for the same comparator set.

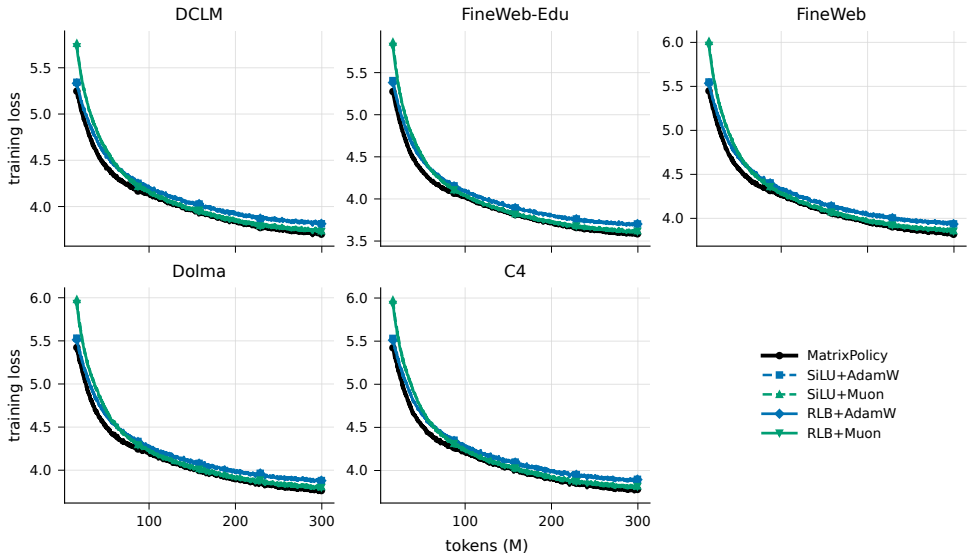

Figure 10: E2 training-loss trajectories for the same comparator set.

B.3 Positive matrix-pair rescaling with an RMS floor

Set $\epsilon_{\text{rms}} = 0$ and take $a > 0$. Then

$$r(az) = ar(z), \qquad u(az) = u(z), \qquad h(az) = ah(z). \tag{43}$$

Scaling $A_{l,g}$ by a and the matching columns of $B_{l,g}$ by a^{-1} therefore leaves the group contribution unchanged. Applying this independently to all groups gives equation 9.

With $\epsilon_{\text{rms}} > 0$, define r_ϵ and u_ϵ by equation 4. For $\rho^2 = m^{-1}\|z\|_2^2$,

$$u_\epsilon(az) = \kappa(a, z)u_\epsilon(z), \qquad \kappa(a, z) = \frac{a\sqrt{\rho^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2\rho^2 + \epsilon_{\text{rms}}}}. \tag{44}$$

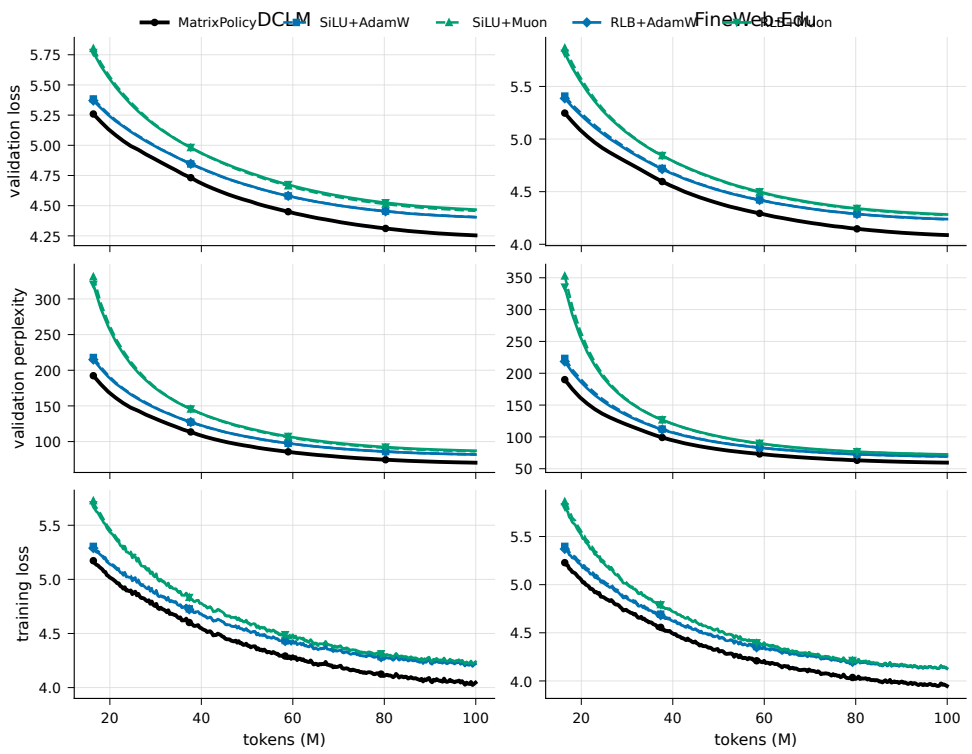

Figure 11: Representative E1 validation-perplexity and training-loss companions on DCLM and FineWeb-Edu, including the appendix RLB optimizer controls. The main text keeps the corresponding SiLU comparison in Figure 4.

Let L_R be a Lipschitz constant of the coordinatewise curve on the segment between $u_\epsilon(z)$ and $\kappa(a,z)u_\epsilon(z)$. After compensating the output matrix by a^{-1}, the group-feature error obeys

$$\left\|a^{-1}h_\epsilon(az) - h_\epsilon(z)\right\|_2 \le \left|\frac{r_\epsilon(az)}{a} - r_\epsilon(z)\right| \left\|R(u_\epsilon(z))\right\|_2 \tag{45}$$

$$+ \frac{r_\epsilon(az)}{a} L_R |\kappa(a,z) - 1| \left\|u_\epsilon(z)\right\|_2 . \tag{46}$$

Both terms vanish when $\epsilon_{\mathrm{rms}} = 0$. When $\rho^2 \gg \epsilon_{\mathrm{rms}}$ and the global response and derivative are bounded on the segment between $u_\epsilon(z)$ and $\kappa(a,z)u_\epsilon(z)$, the bound is small; near the floor, the approximation can be poor. This is the reason MatrixPolicy uses clipped rescaling moves rather than treating matrix-pair balancing as an exact function-preserving operation.

C MatrixPolicy Optimizer Step

D MatrixPolicy Design Rationale

D.1 Role-specific gradients and gain selection

For one training example, let $\bar{y}_l = \nabla_{y_l}\mathcal{L}$ be the upstream gradient, and let $J_{l,g} = \partial h_{l,g}/\partial z_{l,g}$ be the group Jacobian in equation 42. The two matrix gradients have different forms:

$$\nabla_{B_{l,g}}\mathcal{L} = \bar{y}_l h_{l,g}^\top, \qquad \nabla_{A_{l,g}}\mathcal{L} = \left(J_{l,g}^\top B_{l,g}^\top \bar{y}_l\right) x_l^\top . \tag{47}$$

The output matrix sees the realized feature vector $h_{l,g}$ directly. The input matrix sees the upstream gradient filtered through $B_{l,g}$ and the RLB group Jacobian. This motivates using curve-response summaries for the output role and derivative-gain summaries for the input role, while recognizing that these summaries are proxies rather than full matrix preconditioners.

MatrixPolicy update in one optimizer step.

1. Update moving averages of matrix-gradient scales and coefficient-gradient scales from current gradients.
2. Read cached batch-estimated curve gains from the most recent RLB forward path.
3. Step non-matrix parameters, including rational coefficients, with AdamW.
4. Scale RLB matrix gradients with the centered group multipliers in equation 20.
5. Step RLB matrices with the AdamW matrix branch and any active Muon branch in equation 27.
6. Every five optimizer steps, apply the bounded positive matrix-pair rescaling in equation 34.

Figure 12: MatrixPolicy affects only the input and output matrices of RLB sublayers. All other parameters use the AdamW path.

D.2 Scale-normalized pressure and centered redistribution

The exact derivative along the idealized positive-rescaling direction would be

$$\Delta_g = \left\langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \right\rangle_F - \left\langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \right\rangle_F . \tag{48}$$

MatrixPolicy instead uses RMS relative gradient scales because they are stable proxies for how strongly each matrix role is being moved. For nonzero unfloored W, if $W^+ = W - \eta\mathcal{G}$ and η is small, then

$$\log \mathrm{rms}(W^+) - \log \mathrm{rms}(W) = -\eta \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} + O(\eta^2), \tag{49}$$

and Cauchy–Schwarz gives

$$\left| \frac{\langle \mathcal{G}, W \rangle_F}{\|W\|_F^2} \right| \le \frac{\|\mathcal{G}\|_F}{\|W\|_F} = \frac{\mathrm{rms}(\mathcal{G})}{\mathrm{rms}(W)}. \tag{50}$$

The implemented pressures are floored versions of this stable proxy, not natural gradient terms.

Before clipping, the group policy is recentered by

$$\widehat{c}_{l,g,\sigma} = \frac{c_{l,g,\sigma}}{\mathrm{geomean}_{j=1}^{G} c_{l,j,\sigma}}. \tag{51}$$

Therefore

$$\frac{1}{G}\sum_{g=1}^{G} \log \widehat{c}_{l,g,\sigma} = \frac{1}{G}\sum_{g=1}^{G} \log c_{l,g,\sigma} - \log\left(\mathrm{geomean}_{j=1}^{G} c_{l,j,\sigma}\right) = 0. \tag{52}$$

The centered policy redistributes group update scale while preserving the mean log multiplier before clipping. Clipping prevents noisy group statistics from dominating a matrix update, at the cost of slightly perturbing the exact zero-mean log property.

D.3 Bounded matrix-pair balancing and transient direction updates

For the idealized nonzero-matrix calculation, let

$$\gamma_{l,g} = \log \mathrm{rms}(A_{l,g}) - \log \mathrm{rms}(B_{l,g}). \tag{53}$$

The implementation uses the floored ratio in equation 29; near the floor the exact contraction below is only an approximation. On an active positive-rescaling step, $A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}$ and $B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}$, so the log ratio changes as

$$\gamma_{l,g}^{+} = \gamma_{l,g} + 2\ell_{l,g,t}. \tag{54}$$

Ignoring clipping and writing $\lambda_{l,g,t} = s_{l,t} a^{\mathrm{rs}}_{l,g}$, the update in equation 33 gives

$$\gamma^{+}_{l,g} = (1 - \lambda_{l,g,t})\gamma_{l,g} + \lambda_{l,g,t}\tau_{l,g}. \tag{55}$$

Thus, in the nonzero-matrix, unfloored-RMS, $\epsilon_{\mathrm{rms}} = 0$, unclipped idealization, each active rescaling step moves the matrix representative toward the target ratio without changing the represented idealized block map. In the implemented setting this operation runs every five optimizer steps, $\lambda_{l,g,t}$ is bounded by the update schedule and activity factor, and the actual log move is clipped by 0.030.

The transient Muon branch uses the window

$$w(\xi_t) = \mathrm{smoothstep}(0.02, 0.12, \xi_t)\,[1 - \mathrm{smoothstep}(0.20, 0.36, \xi_t)]\,. \tag{56}$$

This confines the matrix-direction branch to the phase in which changes to latent directions most directly affect the coordinates seen by the rational curves. The coefficient-gradient penalty $\psi_{l,t}$ further reduces the branch when the rational coefficients are already moving strongly. The design therefore keeps the branch tied to the state of the RLB sublayer rather than applying a persistent generic matrix rule throughout training.